

RUNPOD EXECUTION ARCHITECTURE

Trustworthy AI for Protein Engineering

Discovering Mutation-Space Reliability and Applying It to Industrial Carbon-Capture Enzyme Design

Complete RunPod Infrastructure, Execution Workflow, GPU Resource Strategy, Checkpoint Architecture, Pipeline Orchestration, Recovery Framework, And Production Operations Guide

Version 2.0

Purpose

This document defines the complete execution architecture used to run the project on RunPod.

The objective is to transform the scientific design defined in the project documents into a reproducible large-scale computational execution system.

This architecture governs:

Infrastructure

GPU Usage

Pipeline Execution

Checkpointing

Recovery

Data Storage

Result Generation

Publication Asset Creation

The system must support complete execution of the project from raw datasets through publication-ready outputs.

RunPod Philosophy

RunPod should not be treated as:

Temporary GPU Rental

RunPod should be treated as:

Scientific Research Infrastructure

Its purpose is to provide a reproducible environment capable of:

Processing Datasets

Running AI Models

Generating Sparsity Metrics

Performing Reliability Analysis

Generating Engineering Recommendations

Producing Publication Assets

High-Level Infrastructure Architecture

Local Workstation



GitHub Repository



RunPod Environment



Scientific Pipeline



Datasets



Analyses



Figures



Reports



Competition Assets



Publication Assets

Infrastructure Components

The execution environment consists of:

Developer Machine

GitHub Repository

RunPod Container

Persistent Storage

GPU Resources

Checkpoint System

Validation Framework

Publication Generation Layer

Recommended Hardware Tiers

Tier 1

Development

Purpose:

Pipeline Development

Debugging

Unit Testing

Recommended:

RTX 4090

24 GB VRAM

Tier 2

Research Production

Purpose:

Full Dataset Processing

ESM Inference

Large Analytical Jobs

Recommended:

A100 40 GB

Tier 3

Large Scale Reprocessing

Purpose:

Complete Rebuilds

Large Mutation Analysis

Future Expansion

Recommended:

H100

RunPod Workspace Structure

Official workspace:

/workspace/trustworthy_ai_protein_engineering/

Root Structure

/workspace/project/

configs/

data/

docs/

src/

tests/

results/

reports/

figures/

science_fair/

papers/

release/

logs/

models/

checkpoints/

Persistent Storage Architecture

Storage must survive:

Pod Shutdown

Restart

Failure Recovery

Persistent Directories

data/

results/

reports/

figures/

release/

checkpoints/

models/

Non-Persistent Directories

temp/

cache/

downloads/

may be rebuilt.

Dataset Architecture

Raw Data Layer

data/raw/

Contains:

Megascale-D

Protein Sequences

Protein Structures

Experimental Epistasis Data

Intermediate Data Layer

data/intermediate/

Contains:

Empirical Sparsity

Evolutionary Sparsity

Structural Sparsity

Final Data Layer

data/final/

Contains:

Combined Sparsity

Experiment Results

Reliability Scores

TKT Analysis

Industrial Fitness Outputs

Complete Execution Pipeline

All project execution follows the official sequence below.

No stage should begin before prior-stage certification.

Phase 1

Infrastructure Validation

Execute:

Environment Validation

Dependency Validation

Storage Validation

GPU Validation

Outputs:

environment_report.md

Phase 2

Dataset Validation

Execute:

Dataset Integrity Checks

Schema Validation

Missing Value Validation

Outputs:

dataset_validation_report.md

Phase 3

Empirical Sparsity Generation

Inputs:

Megascale-D

Outputs:

empirical_sparsity.parquet

Checkpoint:

checkpoints/empirical/

Phase 4

Evolutionary Sparsity Generation

Inputs:

Protein Sequences

ESM Models

Outputs:

evolutionary_sparsity.parquet

Checkpoint:

checkpoints/evolutionary/

GPU Usage

This phase is GPU-intensive.

Preferred execution:

A100

H100

Phase 5

Structural Sparsity Generation

Inputs:

Protein Structures

Outputs:

structural_sparsity.parquet

Checkpoint:

checkpoints/structural/

Phase 6

Combined Sparsity Construction

Inputs:

Empirical

Evolutionary

Structural

Outputs:

combined_sparsity.parquet

Checkpoint:

checkpoints/combined/

Phase 7

Experiment 1

Thermodynamic Inconsistency

Execution

Generate:

Forward Predictions

Reverse Predictions

Antisymmetry Errors

Outputs:

antisymmetry_results.parquet

Checkpoint:

checkpoints/experiment1/

Phase 8

Experiment 2

Prediction Failure Analysis

Execution

Generate:

Experimental Epistasis

Predicted Epistasis

Epistasis Error

Outputs:

epistasis_results.parquet

Checkpoint:

checkpoints/experiment2/

Phase 9

Integrated Reliability Analysis

Inputs:

Combined Sparsity

Experiment 1

Experiment 2

Outputs:

integrated_reliability_analysis.parquet

Checkpoint:

checkpoints/integrated/

Phase 10

Reliability Framework

Generate:

Reliability Scores

Reliability Categories

Outputs:

reliability_scores.parquet

Checkpoint:

checkpoints/reliability/

Phase 11

Transketolase Landscape Generation

Generate:

Complete Single Mutation Space

Outputs:

`tkt_single_mutation_landscape.parquet`

Checkpoint:

`checkpoints/tkt/`

Phase 12

TKT Analysis

Generate:

Stability

Sparsity

Reliability

Outputs:

`tkt_mutation_analysis.parquet`

Phase 13

Active-Site Analysis

Generate:

Distance Metrics

Mechanistic Controls

Outputs:

active_site_analysis.parquet

Phase 14

Industrial Fitness Framework

Generate:

Fitness Scores

Candidate Rankings

Outputs:

industrial_fitness_scores.parquet

top_candidate_mutations.csv

Phase 15

Publication Asset Generation

Generate:

Publication Figures

Statistics Tables

Supplementary Materials

Outputs:

papers/

figures/publication/

Phase 16

Competition Asset Generation

Generate:

Science Fair Figures

Judge Handouts

Board Graphics

Presentation Assets

Outputs:

science_fair/

presentations/

GPU Resource Allocation Strategy

GPU resources should only be used when necessary.

GPU-Intensive Workloads

ESM Inference

Protein LM Scoring

Large Scale Mutation Processing

Bulk Prediction Generation

CPU-Preferred Workloads

Validation

Statistics

Regression

Figure Generation

Report Generation

Publication Packaging

Using CPU for these tasks preserves GPU availability.

Checkpoint Strategy

Every major phase must support recovery.

Checkpoint Layout

checkpoints/

 empirical/

 evolutionary/

 structural/

 combined/

 experiment1/

experiment2/

integrated/

reliability/

tk/

active_site/

industrial_fitness/

Checkpoint Policy

Save after:

Every Completed Batch

Every Completed Protein

Every Major Dataset Write

Failure Recovery Framework

Scenario 1

Pod Shutdown

Recovery:

Restore Latest Checkpoint

Resume Phase

Scenario 2

GPU Failure

Recovery:

Restart Phase

Recover Last Checkpoint

Resume Execution

Scenario 3

Disk Exhaustion

Recovery:

Archive Results

Purge Temporary Files

Resume Execution

Scenario 4

Model Crash

Recovery:

Restore Last Checkpoint

Reduce Batch Size

Restart Job

Logging Architecture

Logs stored in:

logs/

Structure:

logs/

 system/

 datasets/

 experiments/

 validation/

 publication/

Required Log Information

Every execution record must contain:

Timestamp

Module

Dataset

Execution Time

GPU Usage

Memory Usage

Status

Validation Layer

Before completion of every phase:

Schema Validation

Range Validation

Integrity Validation

Output Certification

must pass.

Automated Certification Pipeline

Each phase produces:

Validation Report

Certification Report

Location:

reports/validation/

reports/certification/

Publication Build Architecture

Publication generation occurs only after:

Experiment Certification

Reliability Certification

Engineering Certification

Publication Outputs

Publication Figures

Manuscript Tables

Methods Summaries

Supplementary Statistics

Competition Build Architecture

Competition generation occurs after publication outputs are finalized.

Outputs:

Science Fair Board Graphics

Judge Handouts

Executive Summary

Competition Binder

Presentation Decks

Release Package Generation

Final stage:

release/

Contents:

Datasets

Source Code

Documentation

Figures

Competition Materials

Publication Materials

End-To-End RunPod Workflow

Initialize Pod



Validate Environment



Validate Datasets



Generate Empirical Sparsity



Generate Evolutionary Sparsity



Generate Structural Sparsity



Construct Combined Sparsity



Run Experiment 1



Run Experiment 2



Integrated Reliability Analysis



Generate Reliability Framework



Generate TKT Landscape



Run Active-Site Analysis



Generate Industrial Fitness Rankings



Generate Publication Assets



Generate Competition Assets



Generate Release Package



Archive Results

RunPod Certification Requirements

The RunPod execution architecture is certified only if:

Environment Reproducible



Checkpoints Operational



Recovery Operational



Data Pipeline Operational



Reliability Framework Operational



TKT Analysis Operational



Industrial Fitness Operational



Publication Generation Operational



Competition Generation Operational



Release Packaging Operational



Final Architectural Statement

The RunPod environment exists to execute the complete scientific pathway:

Mutation Space



Sparsity Framework



Prediction Failure



Reliability Estimation



Transketolase Engineering



Industrial Fitness Framework



Industrial Carbon-Capture
Applications

through a fully reproducible, restartable, large-scale computational infrastructure.

END OF RUNPOD EXECUTION ARCHITECTURE

Version 2.0

Approved Production Execution Architecture For The Trustworthy AI For Protein Engineering Project.